

Method to measure DVS-computer-controller latency

Anonymous CVPR 2021 submission

Paper ID 8

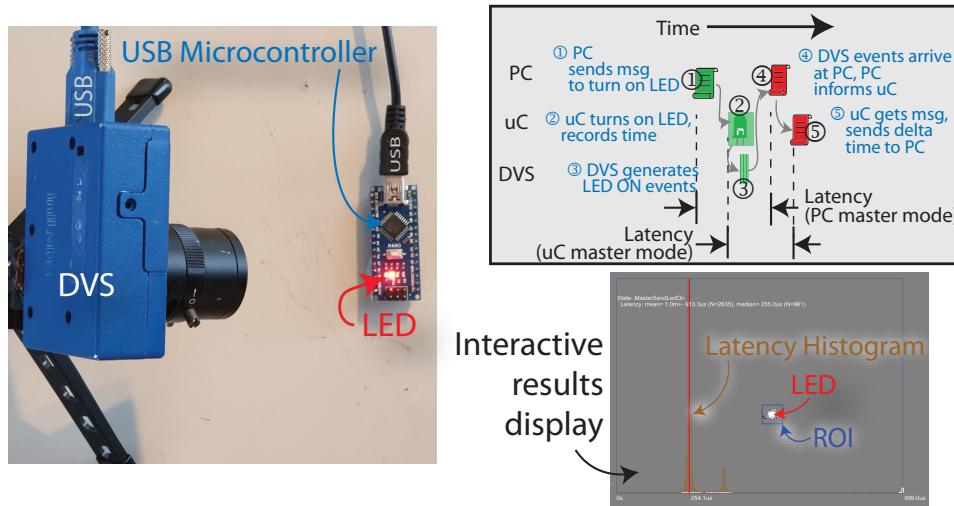


Fig. 1: Measuring DVS-computer-controller latency with LED and Arduino microcontroller.

Abstract

Dynamic vision sensor event cameras respond quickly to visual input, but it is not straightforward to measure the complete latency from DVS visual input to separate controller output. This paper reports a simple method for measuring end-to-end latency by using a light emitting diode that is controlled by a microcontroller connected to the computer. Using a DAVIS346 camera, median latencies below 300 μ s are achievable under Windows 10 with Java host software and USB full-speed controller, which is 10X less than previous results. To minimize latency, it is important to understand the flow of USB data from camera to computer and the influence of the camera's "early-packet" timer.

1. Introduction

Dynamic vision sensor (DVS) event cameras [1], [2] offer superior dynamic range and sparse, quick output that makes them useful for vision under uncontrolled lighting encountered in robotics. Robotic control systems can require minimal end-to-end latency. Measuring end-to-end

camera→computer→controller latency can be done by an oscilloscope connected to the DVS input (LED) and a controller output pin, and [3], [4] reported that latencies of 2-4 ms were possible with DVS cameras connected to PCs running standard operating systems. However, the measurements required an oscilloscope.

2. Method

Fig 1 shows the setup and algorithms steps of one of the LED latency (LEDLAT) measurement algorithms. It uses an Arduino Nano microcontroller and its built-in LED. It can be purchased for about \$3 from Chinese sources. The Arduino has the CH340B USB serial interface. We set the baud rate to maximum of 2Mbaud.

The algorithm runs in a loop as shown in the figure and requires no other components or instruments. In this "uC master mode", the microcontroller (uC) takes an action (turning on LED) and measures the time to receive the confirmation from the computer (PC) that it has received the resulting DVS events. The uC records the time it takes for the PC to tell it that it saw the flash. The uC then sends the time to the PC, which accumulates statistics to update

a displayed histogram. An alternate “PC master mode” has the PC as the timer, telling the uC to turn on the LED and recording the time when the DVS output indicates the LED has turned on.

Each LED flash generates many DVS brightness change events. We use only the ON events from the onset of the LED flash. The LED is determined to be ON when the count of ON events within a user-defined region of interest (ROI) centered on the LED exceeds the threshold count N_{on} . A denoising filter removes uncorrelated background activity events to prevent false detections.

Links to host and firmware code will be included after paper acceptance.

3. Factors influencing DVS-computer latency

Camera to computer and computer output latency depends on many factors and it is not the aim of this paper to study all these factors. However some factors specific to DVS cameras are useful to understand.

3.1. Early packet timer

Inivation DVS cameras include ‘early packet timer’ logic based on [5]. This timer ensures that the USB FIFO on the camera is committed for transport to the host at maximum interval $N_{ept} \times 125 \mu s$, $N_{ept} = 1, 2, \dots, 255$, even if the device FIFO (holding up to 256 events) is not full. N_{ept} is the register setting for the early packet timer. Packets can arrive at the host more frequently than this interval if the FIFO fills up with events before timeout. The default value of $N_{ept} = 8$ results in default maximum delay of 1 ms.

3.2. Influence of processing on USB thread

Many USB event camera software systems, including jAER¹, split the event capture and rendering/processing to separate hardware threads of execution to allow for overlapped camera input and display. The maximum-priority USB thread normally only preprocesses the sensor data and copies to another, larger buffer for processing and rendering in the user processing and rendering thread. However, it is possible to process the data directly in the USB thread for minimum latency. We used this option to obtain the measurement result.

4. Experimental results

Fig. 2 shows a histogram of the our smallest latency results. The median latency is about 250 μs . We obtained these results in PC master mode using $N_{ept} = 1$ (125 μs), and $N_{on} = 15$ events. Table 1 lists results obtained with other settings. The influence of N_{ept} is clear. We do not know why PC master mode results in smaller variance. We

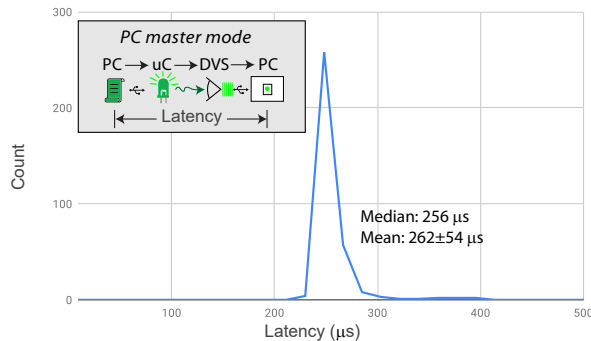


Fig. 2: Measured latency. Settings: $N_{on} = 15$, $N_{ept} = 1$. Obtained with DAVIS346 camera, jAER-1.9.2 commit version 953b6b1 on 7.3.2021, running Windows 10 build 20H2 19042.804, Java 1.8.1, AMD Ryzen 7 3700X 8-core processor 3.59 GHz. Arduino Nano with CH340B USB Serial. Photoreceptor bias current: 2.1nA. Source-follower bias current: 52pA. Thresholds: ± 0.23 log units.

cannot easily determine what part of the latency is from DVS to PC and from PC to uC. Pinging the uC with a single byte and receiving the single byte pong results in about 1 ms total latency, which is likely caused by the USB-2.0 full speed polling interval of 1 ms. This polling interval affects switching from writing to the serial port and then reading from it; apparently isolated writes from PC to uC have a much lower latency, which is why we can send the isolated commands “turn on LED” and “detected LED” without incurring this polling latency. A controller with shorter latency (e.g. based on high-speed or super-speed USB interface) would probably result in shorter latency².

N_{ept}	USB thread	Median (mean $\pm$ std) (us)
1 (PC)	yes	256 (262 $\pm$ 54)
1 (uC)	yes	292 (1500 $\pm$ 480)
4 (PC)	yes	519 (520 $\pm$ 38)
8 (PC)	yes	1000 (1200 $\pm$ 408)
1	no (62 Hz FPS)	13,900 (13,900 $\pm$ 1000)
ping	yes	1000
8	yes	2000-4000 [4]

Table 1: Latency results

5. Discussion and conclusion

LEDLAT measures end-to-end DVS camera-controller latency. The uC board can be purchased for under \$5, and the software measurements make it easy to collect results and experiment with settings. Minimum latencies are more than 10X less than previously reported results [4].

²Tests with Arduino Teensy 4.0 and Portenta H7 Arduino Pro showed very similar results

¹j aerproject.net

References

- [1] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128x128 120dB 15 μ s latency asynchronous temporal contrast vision sensor," *IEEE J. Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Feb. 2008, ISSN: 0018-9200, 1558-173X. DOI: [10.1109/jssc.2007.914337](https://doi.org/10.1109/jssc.2007.914337). [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4444573/>.
- [2] G. Gallego, T. Delbruck, G. M. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. Davison, J. Conradt, K. Daniilidis, and D. Scaramuzza, "Event-based vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–1, 2020, ISSN: 0162-8828, 1939-3539. DOI: [10.1109/TPAMI.2020.3008413](https://doi.org/10.1109/TPAMI.2020.3008413). [Online]. Available: <http://dx.doi.org/10.1109/TPAMI.2020.3008413>.
- [3] T. Delbruck and P. Lichtsteiner, "Fast sensory motor control based on event-based hybrid neuromorphic-procedural system," in *2007 IEEE International Symposium on Circuits and Systems*, New Orleans: IEEE, May 2007, pp. 845–848, ISBN: 9781424409204, 9781424409211. DOI: [10.1109/ISCAS.2007.378038](https://doi.org/10.1109/ISCAS.2007.378038). [Online]. Available: <http://dx.doi.org/10.1109/ISCAS.2007.378038>.
- [4] T. Delbruck and M. Lang, "Robotic goalie with 3 ms reaction time at 4% CPU load using event-based dynamic vision sensor," in *Front. Neurosci.*, vol. 7, p. 223, Nov. 2013, ISSN: 1662-4548, 1662-453X. DOI: [10.3389/fnins.2013.00223](https://doi.org/10.3389/fnins.2013.00223). [Online]. Available: <http://dx.doi.org/10.3389/fnins.2013.00223>.
- [5] R. Berner, T. Delbruck, A. Civit-Balcells, and A. Linares-Barranco, "A 5 meps \$100 USB2.0 Address-Event Monitor-Sequencer interface," in *2007 IEEE International Symposium on Circuits and Systems*, ieeexplore.ieee.org, May 2007, pp. 2451–2454. DOI: [10.1109/ISCAS.2007.378616](https://doi.org/10.1109/ISCAS.2007.378616). [Online]. Available: <http://dx.doi.org/10.1109/ISCAS.2007.378616>.